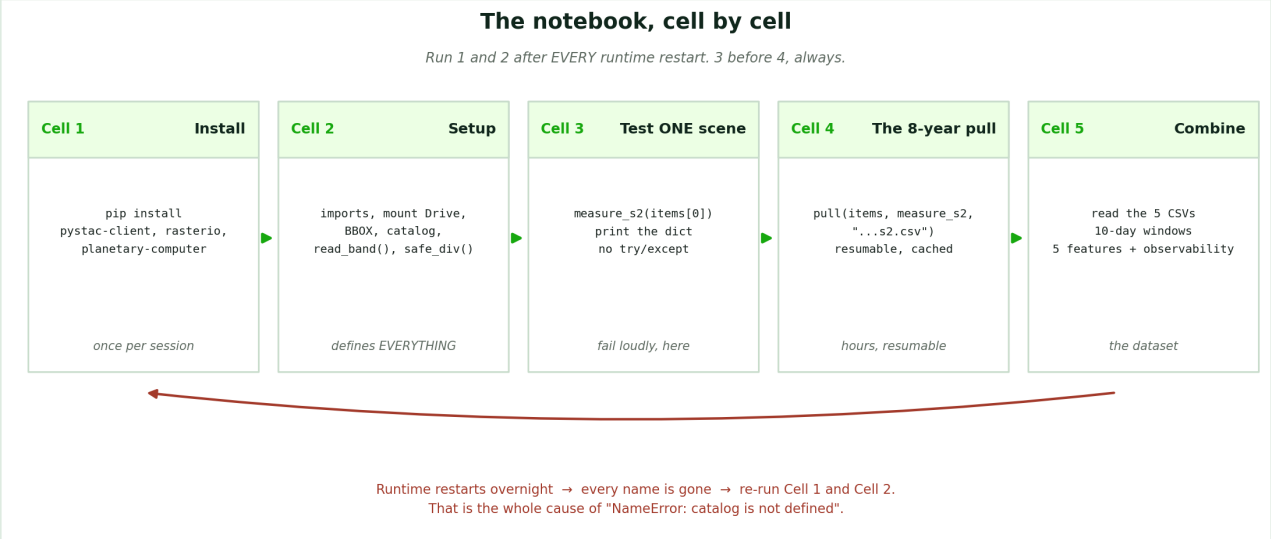
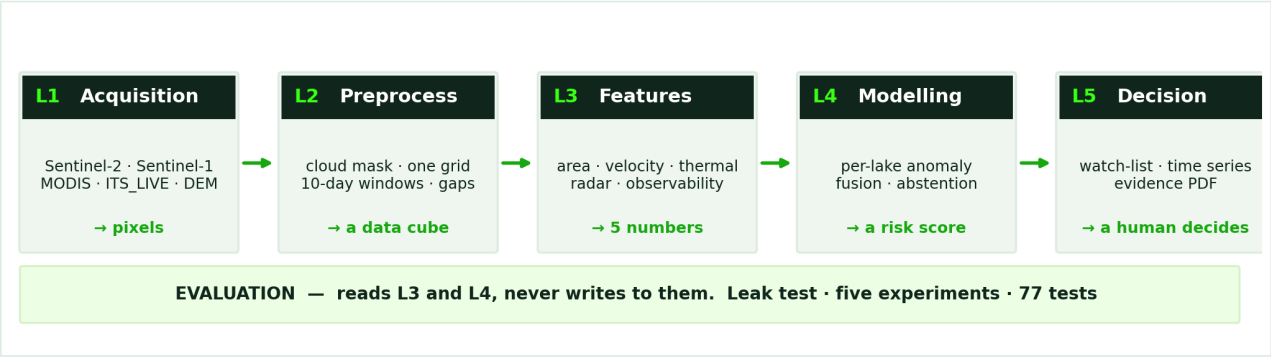
GEWS — the Colab notebook, with pictures

Every cell you need to type, what each one is doing, and what the screen should look like when it works. Five data sources. No account, no API key, no cost.



Five cells, in this order. Cell 2 is the one that defines every name the rest of the notebook uses — which is why a restarted runtime breaks everything at once.

Where these cells sit in the finished system



The notebook is layers L1 and L2. Everything to the right of "a data cube" is a separate job — build the left half first and check it before writing a single line of the model.

Read this before you start. Cell 4 takes hours and is resumable; Cell 3 takes ten seconds and tells you whether Cell 4 is worth starting. Never skip Cell 3.

Cells 1 and 2 — install and setup

Run both after ANY runtime restart, in this order, before anything else.

The five sources this notebook pulls

Source	Where	Pixel	Cadence	What you extract
Sentinel-2 L2A	Planetary Computer STAC	10 m	~5 days	lake area from a water index
Sentinel-1 RTC	Planetary Computer STAC	10 m	~6-12 days	backscatter in dB, dark-pixel area
MODIS 11A1	Planetary Computer STAC	1 km	daily	one mean temperature per window
Copernicus DEM	Planetary Computer STAC	30 m	static	elevation and slope, read once
ITS_LIVE v2	AWS S3 (not STAC)	120 m	annual	glacier speed, one value per year

Only Sentinel-2 uses the "count the water pixels" pattern. The others each reduce differently, which is why they are separate functions rather than one loop.

Cell 1 — Install

```
!pip install -q pystac-client planetary-computer rasterio xarray zarr s3fs
```

Cell 2 — Setup: every name the notebook uses is defined here

```
import os, numpy as np, pandas as pd, matplotlib.pyplot as plt
import rasterio, pystac_client, planetary_computer
from rasterio.windows import from_bounds
from rasterio.warp import transform_bounds

DATA = "/content/gews_data"          # local disk: fast, and no Drive quota
os.makedirs(DATA, exist_ok=True)

BBOX      = [88.18, 27.89, 88.22, 27.93]   # lon_min, lat_min, lon_max, lat_max
SCL_CLOUD = [3, 8, 9, 10]                  # shadow, cloud med, cloud high, cirrus
SCL_WATER = 6
SCL_SNOW  = 11
PIXEL_KM2 = 0.0001                         # one 10 m x 10 m pixel

# GDAL settings that make remote reads fast and stop 403s mid-run
os.environ["GDAL_DISABLE_READDIR_ON_OPEN"]     = "EMPTY_DIR"
os.environ["CPL_VSIL_CURL_ALLOWED_EXTENSIONS"] = ".tif"
os.environ["GDAL_HTTP_MAX_RETRY"]              = "5"
os.environ["GDAL_HTTP_RETRY_DELAY"]            = "2"

catalog = pystac_client.Client.open(
    "https://planetarycomputer.microsoft.com/api/stac/v1")   # NO modifier - see note

def read_band(item, band, bbox, out_shape=None):
    """Read only the pixels inside bbox, straight from the remote file.
       Signed per read, so the token cannot expire half way through a long pull."""
    href = planetary_computer.sign(item.assets[band].href)
    with rasterio.open(href) as src:
        b   = transform_bounds("EPSG:4326", src.crs, *bbox)
        win = from_bounds(*b, transform=src.transform)
        return src.read(1, window=win, out_shape=out_shape).astype("float32")

def safe_div(a, b):
    """Normalised difference without an epsilon in the denominator."""
    out = np.full(a.shape, np.nan, dtype="float32")
    np.divide(a, b, out=out, where=b > 0)
    return out

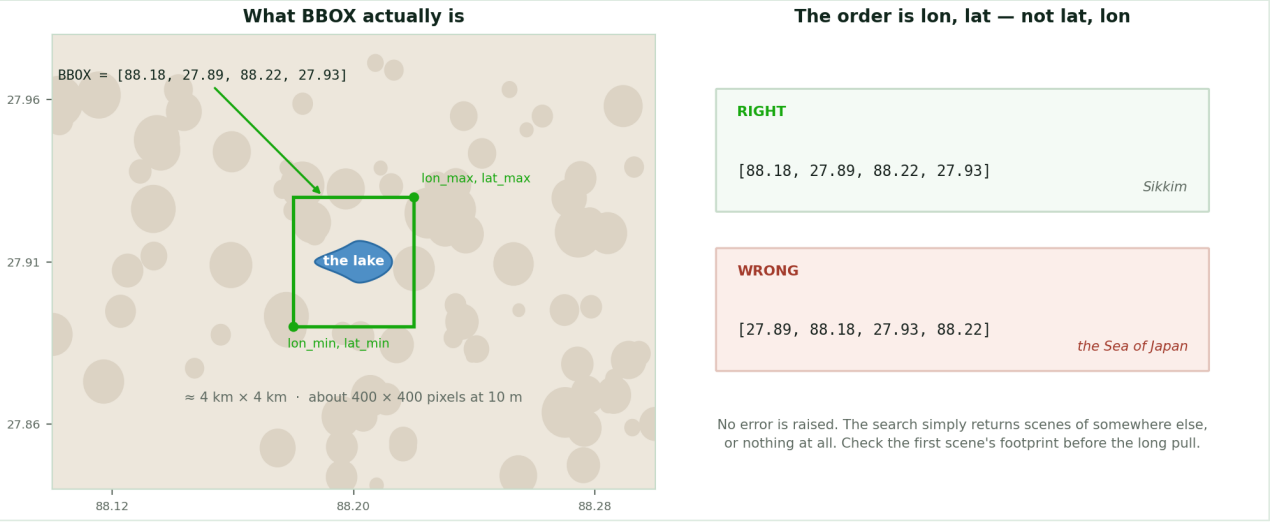
print("setup ok")
```

```
what it should print
setup ok
```

Why no `modifier=sign_inplace`. That signs every scene once, at search time. On a 569-scene pull the last tokens have expired before you reach them and you get `403` around scene 330. Signing inside `read_band` mints a fresh token per read.

BBOX — the one line that decides everything

Get this wrong and nothing errors. You simply measure the wrong place for eight years.



The box is about 4 km across — big enough that the lake never touches an edge, small enough that a read is a few hundred kilobytes instead of a gigabyte.

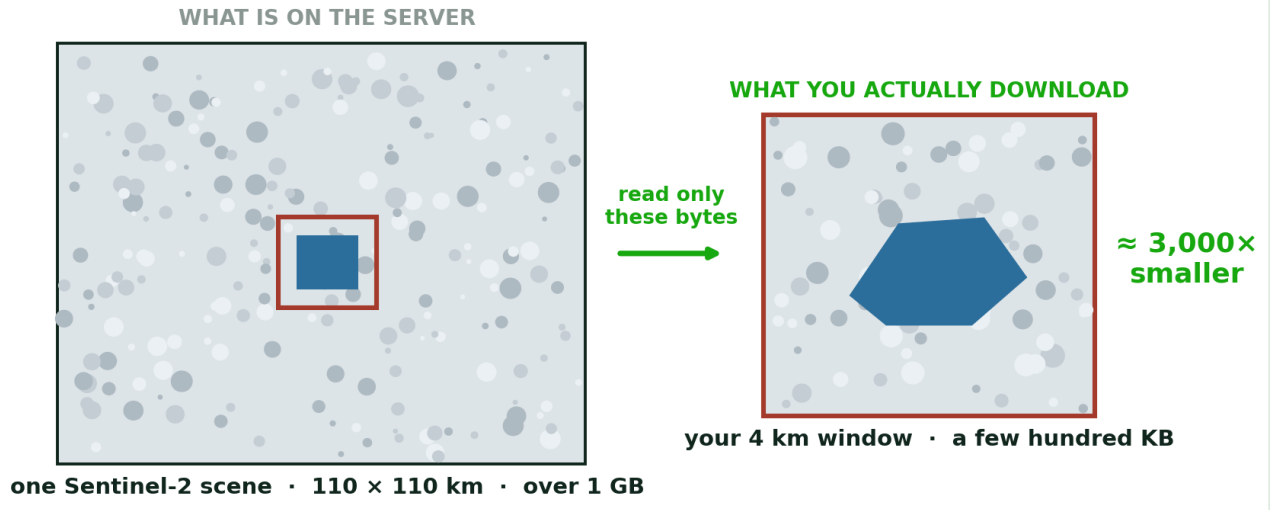
Check the box before the long pull

```
test = list(catalog.search(collections=["sentinel-2-l2a"], bbox=BBOX,
                           datetime="2022-11-01/2022-11-30").items())
print(len(test), "scenes")
print(test[0].properties["s2:mgrs_tile"])     # expect a Sikkim tile: 45RUK / 45RUL
print(test[0].geometry["coordinates"][0][0])  # expect roughly [88.x, 27.x]
```

```
what it should print
4 scenes
45RUL
[88.0021, 27.1058]
```

If the tile is somewhere else, you have the coordinate order wrong. Fix it now — not after a six-hour pull.

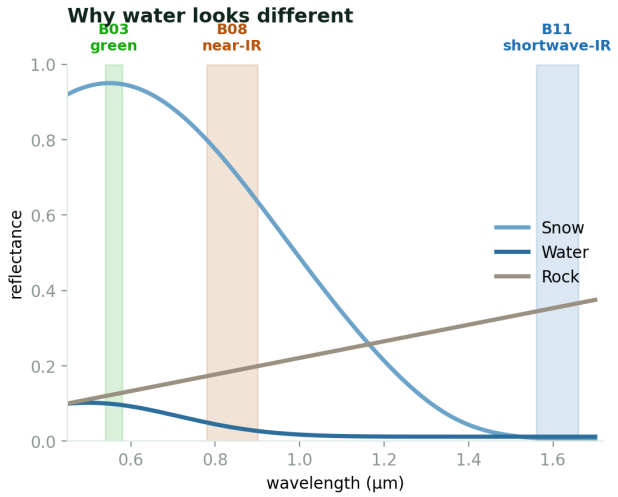
How a windowed read works



The scene stays on Microsoft's server. rasterio asks for only the byte ranges covering your 4 km box — about 3,000 times less data than the whole scene.

1 · Sentinel-2 — lake area

The primary signal. Everything else in the project exists to cover its blind spots.



The two indices

```
NDWI  = (green − nearIR) / (green + nearIR)
MNDWI = (green − shortIR) / (green + shortIR)
```

surface	NDWI	MNDWI	
Water	+0.65	+0.78	high on both
Snow	+0.55	**-0.72**	high on NDWI, LOW on MNDWI
Rock	-0.28	-0.35	low on both

The trap

Snow passes the NDWI test. Only MNDWI rejects it — which is why both must be above 0.30, not just NDWI above 0.

Water absorbs infrared; snow does not — but snow reflects green just as strongly as water does. So snow passes the NDWI test and fails MNDWI, and both must agree.

The measurement function

```
def measure_s2(item, bbox):
    """One optical scene -> area, an independent cross-check, and cloudiness."""
    green = read_band(item, "B03", bbox)
    nir   = read_band(item, "B08", bbox)
    shape = green.shape
    swir  = read_band(item, "B11", bbox, out_shape=shape)   # 20 m -> 10 m
    scl   = read_band(item, "SCL", bbox, out_shape=shape)

    # Reflectance offset: products on baseline 04.00+ (after 25 Jan 2022) carry
    # BOA_ADD_OFFSET = -1000. It does NOT cancel in a normalised index - apply it.
    if item.properties.get("s2:processing_baseline", "00.00") >= "04.00":
        green, nir, swir = green - 1000.0, nir - 1000.0, swir - 1000.0

    ndwi  = safe_div(green - nir,  green + nir)
    mndwi = safe_div(green - swir, green + swir)
    water = (ndwi > 0.30) & (mndwi > 0.30)      # BOTH must agree

    area = float(water.sum()) * PIXEL_KM2
    if area == 0:                # a lake does not vanish - we failed to see it
        area = np.nan

    return {"scene_id": item.id,
            "date": str(item.datetime.date()),
            "area_km2": area,
            "area_scl_km2": float((scl == SCL_WATER).sum()) * PIXEL_KM2,
            "snow_frac":  float((scl == SCL_SNOW).mean()),
            "cloud_frac": float(np.isin(scl, SCL_CLOUD).mean()),
            "baseline": item.properties.get("s2:processing_baseline")}
```

Cell 3 — test ONE scene, and look at it

Ten seconds here saves six hours there.

Run the function once, with no try/except

```
s2 = catalog.search(collections=["sentinel-2-l2a"], bbox=BBOX,
                    datetime="2022-11-01/2022-11-30",
                    query={"eo:cloud_cover": {"lt": 20}})
s2_items = list(s2.items())
print(len(s2_items), "scenes")
print(measure_s2(s2_items[0], BBOX))      # let it fail loudly if it is going to
```

```
what it should print
4 scenes
{'scene_id': 'S2A_MSIL2A_20221104T043831_R033_T45RUL_...',
 'date': '2022-11-04', 'area_km2': 1.6124, 'area_scl_km2': 1.5901,
 'snow_frac': 0.213, 'cloud_frac': 0.041, 'baseline': '04.00'}
```

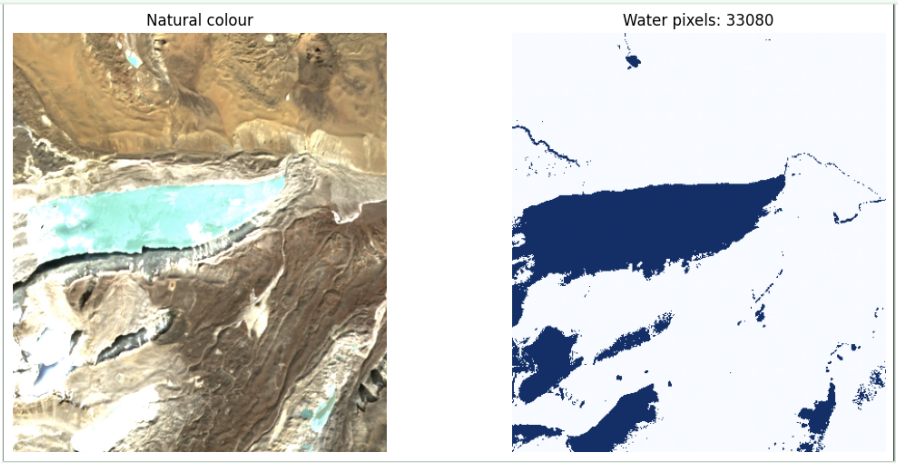
Read the numbers, do not just admire them. `area_km2` and `area_scl_km2` are two independent estimates — yours and ESA's. 1.61 against 1.59 is agreement. 1.6 against 13.4 means your mask is catching snow.

Look at the pixels, not only the number

```
import matplotlib.pyplot as plt
it = s2_items[0]
r, g, b = (read_band(it, x, BBOX) for x in ("B04", "B03", "B02"))
rgb = np.dstack([r, g, b]); rgb = np.clip(rgb / 3000, 0, 1)

nir  = read_band(it, "B08", BBOX)
swir = read_band(it, "B11", BBOX, out_shape=g.shape)
ndwi  = safe_div(g - nir,  g + nir)
mndwi = safe_div(g - swir, g + swir)

f, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(rgb);                      ax[0].set_title("natural colour")
ax[1].imshow(ndwi > 0.0,  cmap="Blues"); ax[1].set_title("NDWI > 0  (the trap)")
ax[2].imshow((ndwi > 0.30) & (mndwi > 0.30), cmap="Blues")
ax[2].set_title("NDWI & MNDWI > 0.30  (what GEWS uses)")
[a.axis("off") for a in ax]; plt.show()
```



A real run of this cell. Left: the lake in natural colour. Right: NDWI > 0 alone — 33,080 "water" pixels, spilling across snow and shaded slopes. That is 3.3 km^2, double the lake.

Cell 4 — the eight-year pull

One resumable helper, reused by every source. Run it, close the laptop, run it again.

The helper

```
STOP_ON_FIRST_ERROR = True          # set False once it runs clean

def pull(items, fn, cache_name):
    cache = os.path.join(DATA, cache_name)
    done  = set(pd.read_csv(cache)["scene_id"]) if os.path.exists(cache) else set()
    print(len(items), "scenes,", len(done), "already cached")

    def flush(rows):
        if rows:
            pd.DataFrame(rows).to_csv(cache, mode="a", index=False,
                                      header=not os.path.exists(cache))
        return []

    rows, failed, first = [], 0, True
    for n, it in enumerate(items):
        if it.id in done:
            continue
        try:
            rows.append(fn(it, BBOX))
        except Exception as e:
            if first and STOP_ON_FIRST_ERROR:
                raise                               # full traceback, first failure
            first, failed = False, failed + 1
            print("skipped", it.id, type(e).__name__, e)   # print e, not just its type
            continue
        if len(rows) >= 10:
            rows = flush(rows); print(n, "/", len(items), "-", failed, "failed")
    flush(rows)
    print("done -", failed, "failed")
    return cache

s2_all = list(catalog.search(collections=["sentinel-2-l2a"], bbox=BBOX,
                             datetime="2017-01-01/2024-12-31",
                             query={"eo:cloud_cover": {"lt": 60}}).items())
CACHE_S2 = pull(s2_all, measure_s2, "south_lhonak_s2.csv")
```

```
what it should print
569 scenes, 0 already cached
10 / 569 - 0 failed
20 / 569 - 0 failed
...
560 / 569 - 3 failed
done - 3 failed
```

A guard worth pasting at the top of Cell 4

```
for _n in ["catalog", "read_band", "pull", "BBOX", "PIXEL_KM2"]:
    if _n not in globals():
        raise RuntimeError(f"'{_n}' is missing - re-run Cell 1 and Cell 2.")
```

This turns "NameError on scene 1 of 569" into one sentence telling you exactly what to do.

Check this before you trust the series: plot area either side of January 2022. If there is a step change, the reflectance offset is why — and a threshold calibrated on one era is wrong for the other.

Sentinel-2 L2A — schema

Collection `sentinel-2-l2a` · surface reflectance, atmospherically corrected.

Assets you can read

Asset	Band	Res.	dtype	Meaning
B02	blue ~490 nm	10 m	uint16	true-colour display
B03	green ~560 nm	10 m	uint16	NDWI and MNDWI, numerator
B04	red ~665 nm	10 m	uint16	true-colour display
B08	NIR ~842 nm	10 m	uint16	NDWI — water absorbs this
B11	SWIR ~1610 nm	20 m	uint16	MNDWI — separates snow from water
B12	SWIR ~2190 nm	20 m	uint16	alternative to B11
SCL	scene classification	20 m	uint8	per-pixel label, see below
visual	RGB composite	10 m	uint8	quick look only, never for measurement

Reflectance scaling — the part that bites

Values are integers, not reflectance: `reflectance = (DN + offset) / 10000`, where offset is 0 below processing baseline 04.00 and **-1000** from 04.00 onward (products after 25 January 2022).

A normalised index divides by the SUM of two bands, so an additive offset does not cancel — it shrinks the denominator and inflates the index. A fixed 0.30 threshold therefore behaves differently before and after January 2022 unless corrected. Read the baseline from `item.properties["s2:processing_baseline"]`.

SCL — the scene classification classes

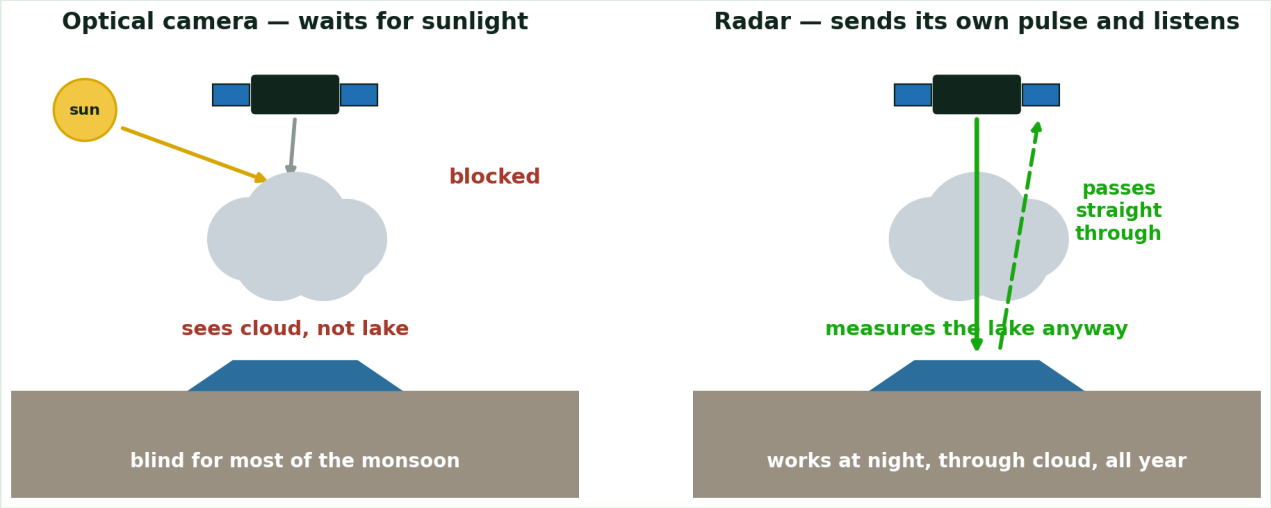
Value	Class	Use
0 / 1	no data / saturated	discard
2	dark area, topographic shadow	discard — shadow can mimic water
3	cloud shadow	counts toward cloud fraction
4 / 5	vegetation / bare soil	land
6	water	ESA's own water flag — your independent cross-check
7	unclassified	treat as unusable
8 / 9	cloud, medium / high probability	counts toward cloud fraction
10	thin cirrus	counts toward cloud fraction
11	snow or ice	why the lake reads frozen in winter

Item properties worth reading

Property	What it gives you
eo:cloud_cover	scene-wide cloud %, for the search filter — not your window's cloudiness
s2:processing_baseline	decides the reflectance offset above
datetime	acquisition time; use .date() for window assignment
s2:mgrs_tile	tile id — a lake near a tile edge appears in two tiles

2 · Sentinel-1 — radar, and the monsoon fix

Collection `sentinel-1-rtc` · radiometrically terrain-corrected gamma-nought.



The single idea behind the project's strongest result: the optical camera waits for sunlight and cloud stops it. Radar sends its own pulse, which passes straight through — at night, in monsoon, all year.

The measurement function

```
def measure_s1(item, bbox):
    """Radar: a smooth water surface reflects away from the sensor and reads DARK."""
    vv = read_band(item, "vv", bbox)
    vh = read_band(item, "vh", bbox, out_shape=vv.shape)

    # RTC ships gamma0 as LINEAR POWER. Decibels = 10 * log10(power).
    vv_db = np.full(vv.shape, np.nan, dtype="float32")
    vh_db = np.full(vh.shape, np.nan, dtype="float32")
    np.log10(vv, out=vv_db, where=vv > 0); vv_db *= 10
    np.log10(vh, out=vh_db, where=vh > 0); vh_db *= 10

    water = vv_db < -18.0        # site-specific - calibrate it, see below

    return {"scene_id": item.id,
            "date": str(item.datetime.date()),
            "vv_db": float(np.nanmean(vv_db)),
            "vh_db": float(np.nanmean(vh_db)),
            "area_sar_km2": float(water.sum()) * PIXEL_KM2,
            "orbit_state": item.properties.get("sat:orbit_state"),
            "rel_orbit":   item.properties.get("sat:relative_orbit")}

s1_all = list(catalog.search(collections=["sentinel-1-rtc"], bbox=BBOX,
                             datetime="2017-01-01/2024-12-31").items())   # no cloud filter
print(len(s1_all), "radar scenes")
print(measure_s1(s1_all[0], BBOX))

CACHE_S1 = pull(s1_all, measure_s1, "south_lhonak_s1.csv")
```

Sentinel-1 — schema, and the three traps

Radar numbers are easy to compute and easy to compute wrongly.

Schema

Asset / property	Type	Units	Notes
vv	float32	gamma0, linear power	co-polarised — the one to use for water
vh	float32	gamma0, linear power	cross-polarised — sensitive to structure
sar:instrument_mode	str	—	IW over land; never mix modes
sat:orbit_state	str	ascending / descending	different geometry, different backscatter
sat:relative_orbit	int	—	compare like with like: one track only
sar:polarizations	list	—	VV+VH over most land

Three things that will catch you out

Issue	What to do
Linear power, not dB	Convert with 10·log10. Averaging linear values then converting is not the same as averaging dB — pick one and stay consistent.
Geometry changes the numbers	Backscatter depends on look angle. Filter to a single `sat:relative_orbit` and one `sat:orbit_state` before comparing dates, or you are measuring the orbit, not the lake.
-18 dB is not universal	Take a date with a clear Sentinel-2 scene nearby, compare the two areas, move the threshold until they agree — then freeze it and record it in config.

Calibrating that threshold — run once, then freeze the answer

```
# Pick a radar scene close in time to a clear optical one, and make them agree.
ref_s2 = s2_items[0]                                  # the clear scene from Cell 3
ref_s1 = min(s1_all, key=lambda i: abs(i.datetime - ref_s2.datetime))
print("gap:", abs(ref_s1.datetime - ref_s2.datetime).days, "days")   # want <= 3

target = measure_s2(ref_s2, BBOX)["area_km2"]
vv_db  = 10 * np.log10(np.clip(read_band(ref_s1, "vv", BBOX), 1e-9, None))

for thr in np.arange(-24, -12, 0.5):
    area = float((vv_db < thr).sum()) * PIXEL_KM2
    print(f"{thr:6.1f} dB -> {area:6.3f} km2   (optical says {target:.3f})")

# Write the winner into your config as a NUMBER WITH A DATE and never tune it again.
# Re-tuning a threshold after you have seen a result is how a null becomes a "success".
```

```
what it should print
gap: 2 days
 -20.0 dB ->  1.104 km2   (optical says 1.612)
 -19.0 dB ->  1.389 km2   (optical says 1.612)
 -18.0 dB ->  1.627 km2   (optical says 1.612)   <- closest
 -17.0 dB ->  2.011 km2   (optical says 1.612)
```

If `sentinel-1-rtc` fails on scene 0 with `CPLE_OpenFailedError`, it is in Planetary Computer's restricted tier. Switch to `sentinel-1-grd` — same vv/vh assets, fully open — and use `20*log10(DN)` with a percentile threshold, because GRD digital numbers are uncalibrated.

3 · MODIS — melt-season temperature

Collection `modis-11A1-061` · daily land surface temperature.

The measurement function

```
def measure_lst(item, bbox):
    """1 km pixels: your 4 km box is about 4x4 of them - take a mean, not a mask."""
    raw = read_band(item, "LST_Day_1km", bbox)
    raw[raw == 0] = np.nan                    # 0 is the fill value
    celsius = raw * 0.02 - 273.15             # scale factor, then kelvin -> Celsius
    return {"scene_id": item.id,
            "date": str(item.datetime.date()),
            "lst_c": float(np.nanmean(celsius)),
            "valid_frac": float(np.isfinite(celsius).mean())}

lst_all = list(catalog.search(collections=["modis-11A1-061"], bbox=BBOX,
                              datetime="2017-01-01/2024-12-31").items())
CACHE_LST = pull(lst_all, measure_lst, "south_lhonak_lst.csv")
```

Asset	dtype	Scale	Units	Fill	Notes
LST_Day_1km	uint16	0.02	kelvin	0	daytime overpass — the one you want
LST_Night_1km	uint16	0.02	kelvin	0	useful for a day-night range
QC_Day	uint8	—	bit flags	—	quality; the low bits are the mandatory QA
Clear_day_cov	uint16	0.0005	fraction	0	how much of the pixel was clear

Delivered in a sinusoidal projection, not lat/lon — `transform_bounds` inside `read_band` already handles that. At 1 km your window is only a handful of pixels, so one cloudy pixel moves the mean noticeably. Keep `valid_frac` and drop low-coverage days.

4 · Copernicus DEM — the shape of the dam

Collection `cop-dem-glo-30` · static, read once per lake.

Read it once and store it

```
dem_item = list(catalog.search(collections=["cop-dem-glo-30"], bbox=BBOX).items())[0]
elev = read_band(dem_item, "data", BBOX)          # metres above the EGM2008 geoid

dy, dx = np.gradient(elev, 30.0)                  # ~30 m spacing
slope_deg = np.degrees(np.arctan(np.hypot(dx, dy)))

static = {"elev_min_m":     float(np.nanmin(elev)),
          "elev_max_m":     float(np.nanmax(elev)),
          "slope_mean_deg": float(np.nanmean(slope_deg)),
          "slope_p90_deg":  float(np.nanpercentile(slope_deg, 90))}
print(static)
```

```
what it should print
{'elev_min_m': 5182.3, 'elev_max_m': 5931.7,
 'slope_mean_deg': 14.8, 'slope_p90_deg': 31.2}
```

The grid is in degrees, so a flat 30 m spacing in `np.gradient` is an approximation that worsens away from the equator. At 28°N it is close enough for a moraine-slope proxy; reproject to UTM first if you need slope to be exact.

5 · ITS_LIVE — glacier surface velocity

The odd one out: not on the STAC catalogue, and annual rather than per-window.

ITS_LIVE publishes Zarr datacubes on anonymous S3 rather than scenes in a catalogue. You open a cube with xarray, slice the region, and reduce over time — there is no per-scene loop, so `pull()` does not apply here.

The shape of the code

```
import xarray as xr
from pyproj import Transformer

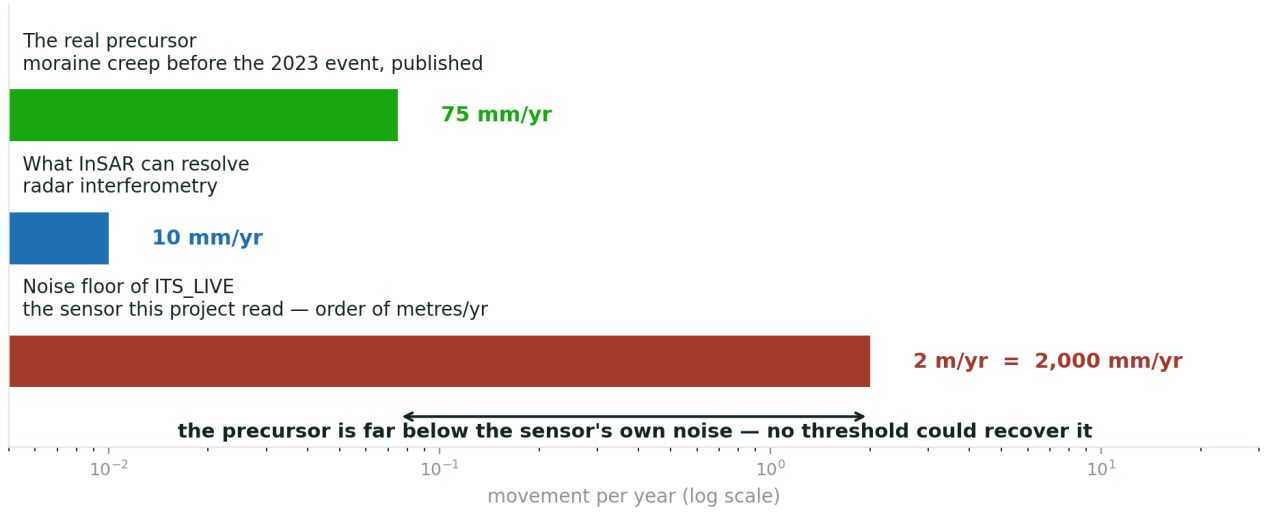
# CUBE_URL: the cube covering 88.2E 27.9N, from the ITS_LIVE catalogue.
# Get the current catalogue location from the ITS_LIVE project site - the path has
# moved between releases and a wrong URL fails in a confusing way.
ds = xr.open_zarr(CUBE_URL, storage_options={"anon": True}, consolidated=True)
print(ds.attrs)          # print this ONCE and use the CRS that is actually there

tf = Transformer.from_crs("EPSG:4326", ds.attrs["projection"], always_xy=True)
x0, y0 = tf.transform(BBOX[0], BBOX[1])
x1, y1 = tf.transform(BBOX[2], BBOX[3])

v = ds["v"].sel(x=slice(min(x0, x1), max(x0, x1)),
                y=slice(max(y0, y1), min(y0, y1)))     # y usually descends

annual = (v.quantile(0.9, dim=["x", "y"])
            .groupby("mid_date.year").max()
            .to_dataframe().reset_index())
annual.to_csv(os.path.join(DATA, "south_lhonak_vel.csv"), index=False)
print(annual)
```

Variable	Units	Dims	Notes
v	m/yr	mid_date, y, x	speed magnitude — the one you want
vx, vy	m/yr	mid_date, y, x	components, if you need direction
v_error	m/yr	mid_date, y, x	per-estimate uncertainty — read it, it is large
mid_date	datetime	—	midpoint of the image pair, irregularly spaced
date_dt	days	mid_date	separation of the image pair



Build this one last, and expect little from it. The documented South Lhonak precursor was 70–80 mm/yr; ITS_LIVE tracks features on 120 m pixels with a noise floor of order metres per year. That null is a finding about the instrument, not a bug in your code.

Cell 5 — five files into one dataset

The step where it stops being satellite data and becomes something a model can read.

What Cell 5 builds: five cached files into one table

One row per lake per 10-day window. Five numbers plus how much of the window you could see.

`south_lhonak_s2.csv` *per scene* → lake_area_km2

`south_lhonak_s1.csv` *per scene* → sar_backscatter_change

`south_lhonak_lst.csv` *per day* → thermal_index

`south_lhonak_vel.csv` *per year* → velocity_trend, _accel

`dem_static.json` *once* → slope, elevation (context)

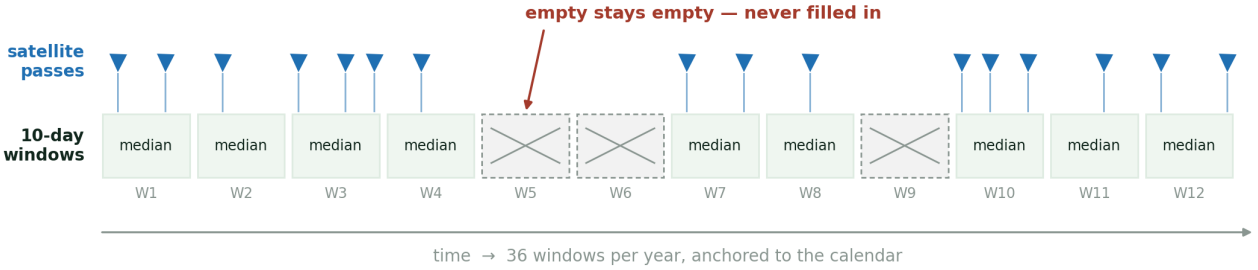
resample to the SAME 10-day grid

window_start	lake_area	vel_trend	vel_accel	thermal	sar_chg	obs
2024-10-01	1.612	18.2	1.1	112.4	-0.31	0.82
2024-10-11	1.618	18.4	1.2	96.0	-0.28	0.77
2024-10-21	NaN	18.4	1.2	88.1	-0.30	0.24
2024-11-01	1.604	18.5	1.1	61.7	-0.35	0.68
2024-11-11	1.599	18.5	1.0	44.2	-0.33	0.71

This table — not the satellite scenes — is what the model reads.

The greyed row: observability 0.24 is below the 0.30 floor
→ that window ABSTAINS. risk = NaN, never 0.0

The common time grid



36 windows a year, anchored to the calendar, identical for every lake and every sensor. Sentinel-2 arrives every ~5 days, Sentinel-1 every ~6–12, MODIS daily — the grid is what lets them be compared at all.

Cell 5 — the combine code

Read the five caches, put them on one grid, write one file.

The combine cell

```
def to_window(dates):
    """Calendar-anchored 10-day windows: 36 per year, identical for every lake."""
    d = pd.to_datetime(dates)
    slot = ((d.dt.dayofyear - 1) // 10).clip(upper=35)
    return pd.to_datetime(d.dt.year.astype(str)) + pd.to_timedelta(slot * 10, "D")

s2  = pd.read_csv(f"{DATA}/south_lhonak_s2.csv")
s1  = pd.read_csv(f"{DATA}/south_lhonak_s1.csv")
lst = pd.read_csv(f"{DATA}/south_lhonak_lst.csv")
vel = pd.read_csv(f"{DATA}/south_lhonak_vel.csv")

for df in (s2, s1, lst):
    df["window"] = to_window(df["date"])

# one row per window per source
g_s2  = s2.groupby("window").agg(lake_area_km2=("area_km2", "median"),
                                 n_s2=("area_km2", "count"))
g_s1  = s1.groupby("window").agg(vv_db=("vv_db", "median"), n_s1=("vv_db", "count"))
g_lst = (lst.assign(dd=lst["lst_c"].clip(lower=0))
            .groupby("window").agg(thermal_index=("dd", "sum"), n_lst=("dd", "count")))

df = g_s2.join(g_s1, how="outer").join(g_lst, how="outer")
df["year"] = df.index.year
df = df.join(vel.set_index("year")[["v"]], on="year")

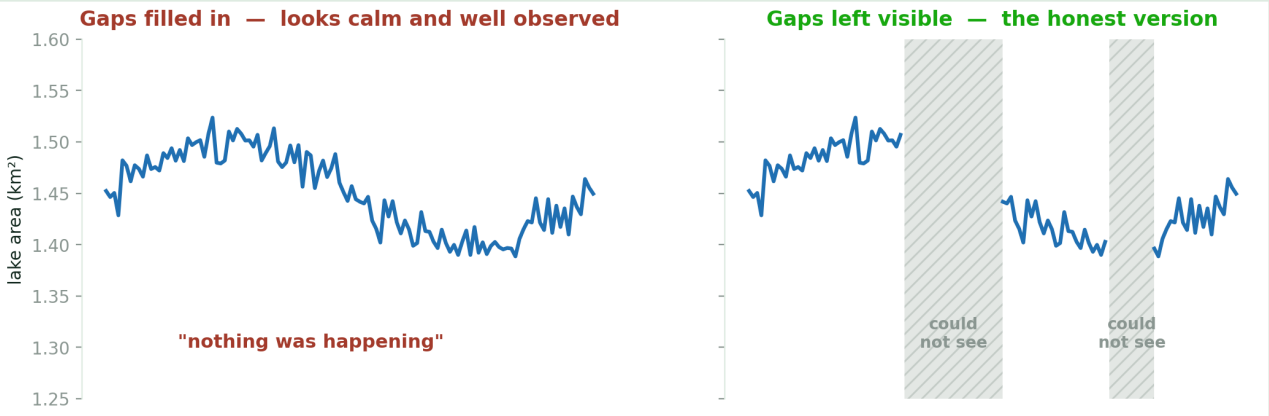
# the two velocity features are differences, not levels
df["velocity_trend"] = df["v"].diff()
df["velocity_accel"] = df["velocity_trend"].diff()
# radar enters as a change, not a level
df["sar_backscatter_change"] = df["vv_db"].diff()

# observability: how much of this window we could actually see
df["observability"] = (0.40 * (df["n_s2"]  > 0).astype(float)
                     + 0.30 * (df["n_s1"]  > 0).astype(float)
                     + 0.15 * (df["n_lst"] > 0).astype(float)
                     + 0.15 * df["v"].notna().astype(float))

FEATURES = ["lake_area_km2", "velocity_trend", "velocity_accel",
            "thermal_index", "sar_backscatter_change"]
final = df[FEATURES + ["observability"]]
final.to_csv(f"{DATA}/south_lhonak_windows.csv")
print(final.tail())
print("windows:", len(final),
      "| assessable:", int((final.observability >= 0.30).sum()))
```

```
what it should print
windows: 288 | assessable: 241
```

Gaps are never filled in. No interpolation, no forward-fill. A window you could not see stays NaN and its observability stays low — because drawing a line across a gap converts "we were blind for 40 days" into "nothing was happening", and those are opposite claims about the safety of a lake.



The same data, two charts. The filled-in version on the left is the one that looks professional and the one that lies.

What each source contributes, and what goes wrong

Keep this page open while the pull runs.

Signal	Units	Comes from	Conditioning before modelling
lake_area_km2	km^2	Sentinel-2 B03/B08/B11	deseasonalised against the lake's own causal climatology
sar_backscatter_change	dB	Sentinel-1 vv	already a difference — not deseasonalised
thermal_index	°C-days	MODIS LST_Day_1km	deseasonalised
velocity_trend	m yr^{-1} per yr	ITS_LIVE v	already a difference
velocity_accel	m yr^{-2}	ITS_LIVE v	already a second difference
(slope, elevation)	degrees, m	Copernicus DEM	static context, not a time signal

The errors that actually happen

Symptom	Cause and fix
`NameError: 'catalog' is not defined`	The runtime restarted and every name is gone. Re-run Cell 1 and Cell 2. Use the guard on page 6.
Lake is in the ocean off Africa	bbox order — it is lon, lat, lon, lat, not latitude first.
`403` around scene 300 of 569	SAS tokens minted at search time have expired. Sign inside `read_band`, as Cell 2 does.
Every scene skipped with the same error	A function is undefined — re-run Cell 2. And print `e`, not just `type(e).__name__`, or you cannot see why.
`CPLE_OpenFailedError` on scene 0 of sentinel-1-rtc	Restricted collection. Use `sentinel-1-grd`, or get a Planetary Computer subscription key.
One read takes 10 minutes	You are pulling the whole scene. The windowed read in `read_band` avoids it.
Step change in the series at Jan 2022	Sentinel-2 reflectance offset for baseline 04.00+. See page 7.
Radar values jump between dates	You are mixing orbit tracks. Filter to one `sat:relative_orbit`.
Temperature around 273	You skipped the 0.02 scale factor, the kelvin conversion, or both.
Area reads 0.00 km^2	Record it as NaN. A lake does not vanish; you failed to see it.

Colab-specific

Problem	What to do
`Saving failed since ...` in every cell	Google Drive is full. Write to `/content/gews_data` instead (Cell 2 does), then `files.download(...)` the CSVs at the end.
Everything vanishes on session end	Local disk is wiped with the runtime. Download the CSVs, or copy them to Drive once the pull finishes.
Session dies after ~90 idle minutes	`pull()` resumes from the cache — just run it again.
Dashboard and tests do not fit a notebook	Move to a local setup once you reach Streamlit and pytest.

The habit worth forming: write down the number you expect before you run the code. If you expect 1.6 km^2 and get 13.4, you have found something. With no expectation, 13.4 is just a number on a screen and you move on.